

Validador de Pedidos

GoCase

Documentação Completa da Automação
Agente de validação e organização de pedidos de fábrica

Business Case — Processo Seletivo de Estágio em RPA
GoCase (GoGroup) — Extrema/MG

Data: 19/07/2026
Área de negócio: Operações de Fábrica

1. Sumário Executivo

Este documento descreve o **Validador de Pedidos GoCase**, uma automação em Python que atua como filtro de qualidade entre a captação de pedidos e o chão de fábrica. O sistema lê a planilha de pedidos exportada do e-commerce, rejeita os pedidos inconsistentes explicando o motivo de cada rejeição, prioriza os pedidos válidos pelo aperto do prazo de entrega e gera relatórios Excel prontos para a equipe de produção agir.

A dor atacada é concreta: em uma operação de produção sob demanda, pedidos com dados quebrados (cliente sem nome, e-mail inválido, quantidade zerada, valor que não fecha, prazo vencido, pedido duplicado) chegam à fábrica e geram retrabalho, desperdício de material personalizado e atraso de entrega. A conferência manual desses dados é lenta, cansativa e sujeita a falha humana.

A solução processa um lote de 50 pedidos em menos de um segundo, com resultado determinístico e auditável. O núcleo é único e o consumo é **centralizado por uma API HTTP** — ninguém instala Python na própria máquina. A mesma lógica é usada por um **frontend web**, por fluxos **low-code** (n8n, Power Automate) e por um **MCP Server** que ainda oferece correção de rejeitados assistida por IA.

Resultado de uma execução de demonstração

Métrica	Valor
Pedidos processados	50
Pedidos válidos	40 (80%)
Pedidos rejeitados	10 (20%)
Tempo de execução	< 0,05 s
Relatórios gerados	3 planilhas Excel + log técnico

2. Contexto de Negócio e a Dor

A GoCase fabrica produtos personalizados sob demanda — cases, mochilas, garrafas, capas. Cada pedido vira uma ordem de produção física e individualizada. Diferente de um estoque padronizado, um pedido com dado errado não é apenas um registro incorreto: é material gasto, hora-máquina perdida e um cliente que não recebe.

De onde vêm os erros

Pedidos entram por múltiplos canais (site, marketplace, B2B, loja física), cada um com seu nível de validação na origem. O resultado é uma planilha heterogênea onde convivem pedidos perfeitos e pedidos com falhas de captação. Os erros mais comuns e custosos são:

- **Cadastro incompleto** — cliente sem nome, e-mail malformatado (impede notificação de rastreio).
- **Quantidade inválida** — zero ou negativa, vinda de erro de importação.

- **Valor incoerente** — valor_total que não bate com quantidade × valor unitário, sinal de erro de cálculo ou fraude.
- **Prazo vencido** — pedido que já entra atrasado e precisa de tratamento imediato, não de produção normal.
- **Duplicidade** — o mesmo pedido importado duas vezes, que produziria o item em dobro.

O custo de não validar

Sem um filtro automático, a conferência é feita a olho, pedido a pedido. Estimando um tempo conservador de conferência manual dos nove pontos de cada pedido, um lote de 50 pedidos consome tempo significativo de um operador — tempo que se repete a cada lote, todo dia. Além do custo de tempo, a conferência manual deixa passar erros sutis, como uma diferença de centavos no valor total ou uma duplicata separada por dezenas de linhas.

3. Objetivo e Escopo

Objetivo

Automatizar a validação e a priorização de pedidos antes da produção, entregando à fábrica apenas pedidos consistentes, já ordenados por urgência, e devolvendo à gestão a lista de pedidos barrados com o motivo exato de cada um.

Dentro do escopo

- Leitura e tipagem da planilha de pedidos.
- Validação por nove regras de negócio, com acúmulo de múltiplos motivos por pedido.
- Classificação e ordenação dos válidos por prioridade de prazo.
- Geração de três relatórios Excel formatados e de um resumo consolidado.
- Operação em dois modos: standalone e MCP Server.

Fora do escopo (evolução futura)

- Escrita de volta no ERP ou no e-commerce (hoje a saída é relatório).
- Notificação ativa por Slack/e-mail (a arquitetura já prevê, ver seção 16).
- Interface web — o operador usa terminal/planilha ou uma IA via MCP.

4. Visão Geral da Solução

A automação é um **pipeline de cinco etapas** orquestrado por um agente. Cada etapa tem responsabilidade única e é isolada das demais: se uma falha, o erro é registrado e as etapas seguintes que ainda fizerem sentido continuam, evitando que um problema pontual derrube o lote inteiro.

Etapa	O que faz	Módulo
1. Leitura	Lê o Excel e tipa as colunas (datas, números)	leitor.py
2. Validação	Aplica as 9 regras; separa válidos de rejeitados	validador.py
3. Organização	Calcula prioridade e ordena os válidos	organizador.py

Etapa	O que faz	Módulo
4. Relatórios	Gera os 3 Excel formatados	relatorio.py
5. Notificação	Monta e registra o resumo da execução	agente.py

O fluxo de dados é direto: a planilha vira um DataFrame; a validação o divide em `df_validos` e `df_rejeitados` (este último ganha a coluna `motivo_rejeicao`); a organização enriquece os válidos com `dias_restantes` e `prioridade`; e os relatórios materializam tudo em disco.

5. Arquitetura

O projeto segue responsabilidade única por módulo — cada arquivo faz uma coisa e é testável isoladamente. Essa separação foi escolhida para que a evolução futura (trocar a fonte de dados, adicionar notificações) toque apenas um ponto do código.

Módulo	Responsabilidade
config.py	Carrega config.yaml (regras externas) com fallback embutido; nunca quebra.
gerar_dados.py	Fixture de teste/demo: gera planilha simulada com erros propositalis. NÃO é produção.
leitor.py	Lê o Excel, tipa colunas e valida a presença das colunas esperadas.
validador.py	Aplica as 9 regras; separa válidos de rejeitados; acumula motivos.
organizador.py	Calcula dias_restantes e prioridade; ordena os válidos.
relatorio.py	Gera os 3 Excel formatados (cores, bordas, freeze, larguras).
assistente_ia.py	Camada de IA: prepara rejeitados e aplica correções propostas (ver seção 11).
agente.py	Orquestra o fluxo, configura logging e monta o resumo.
api.py	API HTTP (FastAPI): expõe o pipeline a qualquer cliente HTTP.
app_streamlit.py	Frontend web que consome a API (upload/download no navegador).
main.py	Entrada terminal (dev/testes).
mcp_server.py	Entrada MCP Server (stdio, 5 tools + 1 prompt).

Fonte única de verdade

As métricas do resumo são montadas uma única vez pelo agente e reusadas pelo relatório, pelo log e pelo cache MCP — sem recálculo em dois lugares. Os nomes, a ordem e as cores das faixas de prioridade vivem apenas no `config.yaml`: renomear ou recolorir uma faixa reflete em todo o sistema, sem código hardcoded. A extensão para novas fontes (Google Sheets, ERP) e notificações (Slack, e-mail) se dá trocando a leitura/saída no agente, sem tocar as regras — hoje sem camada de abstração especulativa, adicionável quando a necessidade for real.

6. Regras de Validação

Cada pedido é avaliado contra **todas** as regras — um pedido pode acumular vários motivos, concatenados por “; ” na coluna `motivo_rejeicao`. Isso dá à gestão a lista completa de problemas de cada pedido de uma só vez, em vez de um erro por vez a cada reprocessamento.

#	Campo	Regra
1	<code>id_pedido</code>	Não vazio e não duplicado. Na duplicata, a 2ª ocorrência é rejeitada.
2	<code>cliente</code>	Não vazio nem só espaços.
3	<code>email</code>	Formato válido: <code>texto@texto.dominio</code> .
4	<code>quantidade</code>	Número inteiro positivo (> 0).
5	<code>valor_unitario</code>	Número positivo (> 0).
6	<code>valor_total</code>	Bate com $\text{quantidade} \times \text{valor_unitario}$ (tolerância R\$ 0,02).
7	<code>prazo_entrega</code>	Não pode estar no passado.
8	<code>produto</code>	Não vazio.
9	<code>sku</code>	Não vazio.

Exemplo de motivo com múltiplos erros: Cliente vazio; Email inválido: formato incorreto.

7. Classificação de Prioridade

Os pedidos válidos recebem $\text{dias_restantes} = (\text{prazo_entrega} - \text{hoje})$ e são classificados em quatro faixas. A ordenação final coloca os URGENTE primeiro e, dentro de cada faixa, o prazo mais apertado antes — a fila que a fábrica deve seguir.

Prioridade	Dias até o prazo	Cor no relatório
URGENTE	0 a 2 dias	Vermelho claro
ALTA	3 a 5 dias	Laranja claro
NORMAL	6 a 10 dias	Verde claro
BAIXA	11 dias ou mais	Sem cor

8. Formato de Dados

Entrada — `data/pedidos_entrada.xlsx`

Uma linha por pedido, com as colunas abaixo. A falta de qualquer coluna faz o leitor parar cedo com mensagem clara dizendo qual coluna faltou.

Coluna	Tipo	Exemplo
<code>id_pedido</code>	texto	PED-00001

Coluna	Tipo	Exemplo
data_pedido	data	14/07/2026
cliente	texto	Ana Carolina
email	texto	ana.carolina@gmail.com
produto	texto	Case iPhone 15
sku	texto	CSE-IPH15-4821
quantidade	inteiro	3
valor_unitario	número	89,90
valor_total	número	269,70
canal	texto	site
endereço	texto	Rua das Flores, 123 — São Paulo/SP
prazo_entrega	data	25/07/2026

Saída — pasta output/

Para o operador — as três planilhas que interessam ao chão de fábrica:

- **pedidos_validados.xlsx** — aprovados, linhas coloridas por prioridade, valores formatados, cabeçalho congelado.
- **pedidos_rejeitados.xlsx** — com erro, coluna motivo_rejeicao em vermelho/negrito.
- **resumo_execucao.xlsx** — métricas consolidadas (totais, %, por prioridade, por canal, valores, tempo).

Artefatos internos — isolados em output/_sistema/, fora da visão do operador: o log técnico da execução e um cache JSON do último resumo (usado pela tool MCP consultar_resumo). Assim o operador abre output/ e vê apenas as três planilhas que importam.

9. Modos de Operação

O núcleo de validação é único; o consumo é **centralizado** pela API — ninguém instala Python ou bibliotecas na própria máquina. Quatro formas de uso partilham a mesma lógica.

9.1. API HTTP (FastAPI) — núcleo de serviço

```
uvicorn api:app --host 0.0.0.0 --port 8000
```

A API roda num servidor/VM/nuvem e recebe a planilha por HTTP. Endpoints:

Rota	Função
POST /validar	Envia a planilha (multipart) e recebe o resumo em JSON + job_id.
POST /validar-base64	Igual, mas o arquivo vai em base64 num JSON (ideal p/ Power Automate).

Rota	Função
GET /download/{job_id}	Baixa os 3 relatórios num .zip.

Os relatórios de cada job vivem por 1 hora e são apagados automaticamente (TTL, configurável por JOBS_TTL_SEGUNDOS): o servidor não acumula arquivos temporários. A limpeza é preguiçosa — roda a cada nova validação, sem thread nem agendador.

9.2. Frontend web (Streamlit)

```
API_URL=http://SERVIDOR:8000 streamlit run app_streamlit.py
```

Frente web que consome a API. O operador abre no navegador, sobe a planilha, vê as métricas e baixa os relatórios — sem instalar nada na máquina.

9.3. Low-code (n8n, Power Automate, Make)

Um fluxo low-code chama a API ao chegar um arquivo (ex.: nova planilha no SharePoint). n8n é o caminho principal na GoCase: usa POST /validar (multipart); o Power Automate usa POST /validar-base64 (JSON, sem montar multipart). A pasta integracoes/ traz um workflow n8n importável e o guia do Power Automate. Nenhuma dependência no cliente.

9.4. MCP Server (integração com IA)

```
python mcp_server.py
```

O servidor fala o protocolo MCP (stdio, JSON-RPC) e expõe cinco ferramentas e um prompt guia, chamáveis por qualquer IA compatível (Claude Desktop, Cursor, n8n):

Tool	Função
gerar_dados_exemplo	Gera a planilha simulada (uso de teste/demo).
validar_pedidos	Executa o fluxo completo e retorna o resumo formatado.
consultar_resumo	Retorna o resumo da última execução, sem reprocessar.
analisar_rejeitados	Prepara os rejeitados para a IA corrigir (ver seção 11).
revalidar_com_correcoes	Aplica as correções da IA e revalida o lote.

Conectado ao Claude Desktop, o operador pede em linguagem natural — “valide os pedidos da planilha” — e a IA aciona a tool. As duas últimas tools formam a camada de correção assistida detalhada na seção 11.

Para desenvolvimento e testes locais ainda existe o modo terminal (python main.py), mas o uso real é centralizado pela API.

10. Configuração Externa (config.yaml)

As regras de negócio ficam **fora do código**, num arquivo config.yaml. Um gestor não-técnico ajusta limites sem abrir Python nem pedir um redeploy — muda o YAML e roda de novo.

O que é configurável

- **tolerancia_valor** — diferença aceita entre `valor_total` e `quantidade × valor_unitario` (padrão R\$ 0,02).
- **regex_email** — padrão de validação do e-mail.
- **colunas_obrigatorias** — quais colunas a planilha deve conter.
- **faixas de prioridade** — os intervalos de dias de URGENTE, ALTA, NORMAL e BAIXA, e a própria ordem da fila.

Segurança por padrão: se o arquivo faltar ou uma chave estiver ausente, o sistema cai nos valores-padrão embutidos em `src/config.py` e registra um aviso — nunca quebra por configuração incompleta. Assim, ajustar uma regra é seguro e reversível: basta editar o YAML.

11. Camada de IA — Correção Assistida via MCP

Além de barrar pedidos, o sistema ajuda a **recuperá-los**. A camada de IA fecha o ciclo **rejeição** → **correção** → **aprovação** usando a IA já conectada via MCP como o cérebro do processo.

Princípio: sem modelo embutido, sem chave de API

O servidor não embute um LLM próprio nem exige credenciais. Ele prepara os pedidos rejeitados num formato que a IA conectada (Claude Desktop, por exemplo) consegue interpretar, e aceita de volta as correções propostas. A inteligência vive no cliente MCP; o servidor fornece contexto e revalida.

Divisão de trabalho

- **Erros mecânicos** (`valor_total` que não fecha, espaços sobrando, e-mail a normalizar) recebem uma sugestão determinística, resolvida por regra — sem IA.
- **Erros semânticos** (nome digitado errado, e-mail com domínio faltando, quantidade implausível) ficam para a IA, que infere a intenção.
- **Dados ausentes por completo** (cliente totalmente vazio) são sinalizados para revisão humana, nunca inventados.

O fluxo (`tools analisar_rejeitados` e `revalidar_com_correcoes`)

1. A IA chama `analisar_rejeitados` e recebe cada pedido barrado, seus motivos e as sugestões mecânicas já resolvidas.
2. A IA propõe as correções semânticas.
3. A IA envia tudo para `revalidar_com_correcoes`, que aplica as correções à planilha, revalida o lote inteiro e relata quantos pedidos foram recuperados. O prompt `corrigir_pedidos_rejeitados` orchestra esse roteiro.

Exemplo real medido: dois e-mails inválidos corrigidos pela IA reduziram os rejeitados de 10 para 8 e elevaram os válidos de 40 para 42, num único ciclo.

Degradação graciosa: se a IA ou o MCP não estiverem disponíveis, nada quebra — os rejeitados continuam saindo no Excel com o motivo, e a gestão corrige manualmente como antes. A camada de IA é um ganho aditivo, não uma dependência do núcleo.

12. Instalação e Uso — Passo a Passo

O ambiente é preparado **uma vez, no servidor** que hospeda a API. Os usuários finais não instalam nada — só acessam o frontend web ou disparam o fluxo low-code.

No servidor:

1. Instalar Python 3.10+.
2. `pip install -r requirements.txt`
3. (Opcional) Ajustar regras em `config.yaml`.
4. Subir a API: `uvicorn api:app --host 0.0.0.0 --port 8000`
5. (Opcional) Subir o frontend: `streamlit run app_streamlit.py`

Para o usuário final: abrir o frontend no navegador e subir a planilha — ou deixar o fluxo n8n/Power Automate chamar a API automaticamente.

As pastas `data/` e `output/` são criadas automaticamente pelo código — não precisam existir de antemão nem ser versionadas.

13. Decisões de Design

Escolhas de engenharia que não são óbvias apenas lendo o código, registradas aqui para o avaliador entender o raciocínio por trás delas:

- **Log em `stderr`, não `stdout`.** No modo MCP o `stdout` carrega o protocolo JSON-RPC; qualquer log ali corromperia a comunicação e quebraria o cliente. O `stderr` fica livre para diagnóstico e o console segue legível.
- **Reconfiguração para UTF-8.** O console Windows usa `cp1252` e corrompe acentos; os streams são reconfigurados para UTF-8, e os arquivos de log já são gravados em UTF-8.
- **Prazo contado a partir de hoje.** Se o prazo fosse `data_pedido + N dias`, pedidos antigos cairiam no passado por acidente. Contar de hoje mantém a coerência e distribui as faixas de prioridade.
- **Tolerância de R\$ 0,02 no `valor_total`.** Soma de floats acumula erro de arredondamento; sem a tolerância, pedidos corretos gerariam falso positivo.
- **Tipagem com `errors='coerce'`.** Valores inválidos viram `NaN/NaT` na leitura em vez de estourar — a decisão de rejeitar fica com o validador, não com o leitor.
- **Fixture de dados separada da produção.** O gerador de exemplo é ferramenta de teste e não tem lançador próprio, para o operador nunca sobrescrever dados reais com dados falsos por engano.
- **Fonte única de verdade.** Métricas do resumo montadas uma vez; nomes, ordem e cores de prioridade só no `config.yaml` — sem duplicação nem hardcoded espalhado.
- **Regras externas em `config.yaml`.** Limites e faixas fora do código, para o gestor ajustar sem redeploy, com fallback embutido que impede quebra por config faltante.
- **IA como cliente, não como dependência.** A camada de correção usa a IA conectada via MCP; o núcleo funciona igual sem ela (degradação graciosa).

14. Testes e Qualidade

O arquivo `testar.py` roda o fluxo completo e verifica oito pontos, servindo como teste de fumaça antes de qualquer entrega:

- Planilha de entrada gerada (50 pedidos).

- Agente executa sem exceção.
- pedidos_validados.xlsx existe e tem linhas.
- pedidos_rejeitados.xlsx existe e tem linhas.
- resumo_execucao.xlsx existe.
- log_execucao.log existe e não está vazio.
- Consistência: válidos + rejeitados = total processado.
- Todos os rejeitados têm motivo de rejeição preenchido.

Padrões de código aplicados em todo o projeto: Python 3.10+, encoding UTF-8 em toda operação de arquivo, type hints em todas as funções, docstrings em português, logging consistente (sem `print()` solto fora do teste) e `pathlib.Path` em vez de strings de caminho.

15. Ganhos Esperados

Dimensão	Antes (manual)	Depois (automação)
Tempo por lote de 50 pedidos	Dezenas de minutos de conferência	Menos de 1 segundo
Consistência	Varia com o operador e o cansaço	Determinística e repetível
Erros sutis (centavos, duplicata distante)	Passam despercebidos	Detectados por regra
Rastreabilidade	Sem registro do motivo	Motivo explícito por pedido + log
Priorização de produção	Feita a olho	Automática, por aperto de prazo
Recuperação de rejeitados	Correção manual, um a um	Correção assistida por IA via MCP

Além do ganho de tempo direto, a automação **libera o operador** da conferência repetitiva para tarefas de maior valor, **reduz desperdício** de material ao barrar pedidos quebrados antes da produção, e **dá à gestão visibilidade** imediata sobre a qualidade dos pedidos por canal.

16. Limitações e Evolução Futura

Já implementado nesta versão

- **Consumo centralizado por API HTTP** (FastAPI) — zero instalação no cliente.
- **Frontend web** (Streamlit) e **integrações low-code** (n8n, Power Automate) consumindo a mesma API; workflow n8n importável em `integracoes/`.
- **TTL de limpeza** — jobs temporários apagados após 1h, sem acúmulo de arquivos.
- **Configuração externa** (config.yaml) — regras editáveis sem tocar no código.
- **Camada de IA** — correção assistida de rejeitados via MCP, com degradação graciosa.

Limitações atuais

- A entrada é um arquivo Excel; ainda não lê direto do ERP/e-commerce.

- A saída é relatório; não escreve de volta nos sistemas de origem.
- A API é aberta — para expor além da rede interna, falta autenticação (API key).

Roadmap de evolução

- **Autenticação** — API key por header quando a API for exposta fora da rede interna.
- **Conectores de entrada** — ler pedidos direto de Google Sheets, ERP ou banco, trocando a leitura no agente.
- **Notificação ativa** — enviar o resumo e alertas por Slack/e-mail.
- **Escrita de retorno** — marcar no sistema de origem o status de cada pedido.
- **Estado entre execuções** — deduplicar entre lotes e guardar histórico de qualidade.

Documento gerado para o business case do processo seletivo de Estágio em RPA — GoCase (GoGroup). O plano de ação para implementação em produção (cronograma, custos e antecipação de desafios) é entregue em documento à parte.